

Pregunta 3

Sin responder aún

Puntúa como 40,00

⚑ Marcar pregunta

1.- ¿Puede un sistema detectar inanición? Justifique su respuesta, si es Sí explique cómo lo puede realizar, si es No explique cómo puede manejar este problema.

2.- Considere el siguiente código para testear locks y variables de condición. Comienza cuando el hilo "principal" llama a ThreadTest (). Realice una traza de ejecución de este programa hasta que imprima el mensaje "Pare aquí". Considere que la planificación es FCFS y las colas se administran utilizando FIFO.

Lock *l;

Condition *cv;

<pre>void A(int arg) { l->Acquire(); cv->Signal(); Thread->Yield(); cv->Wait(); l->Release(); } void B(int arg) { l->Acquire(); cv->Wait(); Thread->Yield(); cv->Signal(); l->Release(); }</pre>	<pre>void ThreadTest() { Thread *t; l = new Lock("lock"); cv = new Condition("cv"); createThread("A"); createThread("B"); Thread->Yield(); Thread->Yield(); printf("Pare aquí\n"); }</pre>
---	--

Thread->yield obliga al hilo que realiza la llamada a renunciar al uso del procesador y a esperar en la cola de listos.

1. Escribir la secuencia de cambios de contexto que ocurrieron hasta este punto,
2. Enumerar las colas en las que se encuentran los hilos en este punto, y su orden relativo si hay más de un hilo en una cola
3. Indicar el estado de cada uno de los hilos.

1.- Considere un proceso compuesto por dos hilos, p y q , que están compuestos por un conjunto de sentencias atómicas (A, B, C, D, y E). El proceso crea los dos hilos y comienza la ejecución concurrente. Muestre todos los posibles entrelazados de ejecución, considerando que los hilos contienen las siguientes sentencias.

<pre>void p() { A; B; C; }</pre>	<pre>void q() { D; E; }</pre>
--	---------------------------------------

2.- ¿Puede un sistema detectar inanición? Justifique su respuesta, si es Sí explique cómo lo puede realizar, si es No explique cómo puede manejar este problema.

1. Si tiene un sistema operativo que está diseñado para una sola CPU y desea adaptarlo a un sistema de múltiples CPU, describa los tipos de cambios / adiciones al software o hardware que necesitaría realizar en relación con el primitivas de sincronización (bloqueos, etc...) y por qué. Liste tres.

1.- ¿Por qué la planificación de colas multinivel con retroalimentación varían el quantum de tiempo asociado con diferentes colas? ¿Cuál sería la desventaja de tener el mismo quantum de tiempo en todos los niveles?

2.- ¿Qué es falso acerca del paso de mensajes? Justifique su respuesta

1. Implica una sección crítica en los datos del mensaje.
2. Una recepción bloqueante implica sincronización con el emisor.
3. Un envío bloqueante permite al emisor utilizar el buffer del mensaje cuando se desbloquea.
4. El paso de mensajes se implementa siempre con memoria compartida.

1) La planificación de colas multinivel con retroalimentación varía el quantum de tiempo asociado a las diferentes colas para atender con mayor prioridad a los procesos mas cortos (ej. aquellos que requieren de una E/S o que requieren poco tiempo de CPU y se ejecutan con mucha frecuencia), esto ocurre porque las colas de mayor prioridad tienen un quantum lo suficientemente chico para que los procesos que requieran menor tiempo de CPU completen su ejecución, mientras que los que requieran mayor tiempo caen a un nivel de menor prioridad. Teniendo el mismo quantum de tiempo en todos los niveles no habría esta distinción de priorizar a los procesos que requieren menor tiempo de CPU y/o se ejecutan con mayor frecuencia.

2) 4. Falso. Los pasos de mensajes, además de implementarse con memoria compartida, es posible implementarlos con mecanismos de comunicación directa o indirecta (mailbox). Otra forma es haciendo uso de pipes.

1.- Considere un proceso compuesto por dos hilos, p y q , que están compuestos por un conjunto de sentencias atómicas (A, B, C, D, y E). El proceso crea los dos hilos y comienza la ejecución concurrente. Muestre todos los posibles entrelazados de ejecución, considerando que los hilos contienen las siguientes sentencias.

void p()	void q()
{ A;	{ D;
 B;	 E;
 C;	}
}	

2.- ¿Puede un sistema detectar inanición? Justifique su respuesta, si es Sí explique cómo lo puede realizar, si es No explique cómo puede manejar este problema.

1)

2) No no es posible detectar la inanición en un sistema, es por esto que se debe evitar implementando mecanismos como, por ejemplo, en la planificación por prioridad, aumentar la prioridad de un proceso a

medida que pasa el tiempo y no es atendido (envejecimiento). En el caso del mecanismos de recuperación de interbloqueo "apropiación de recursos", para evitar la inanición de un proceso por ser elegido constantemente como víctima para hacerle rollback, se podría incorporar la cantidad de rollbacks hechos a un proceso como factor a la hora de elegir una víctima logrando que no sea un mismo proceso el que siempre se elige.

[Parcial Fausto ejercicio 2 CPUs.xlsx - Hojas de cálculo de Google](#)

1. Describa las diferencias entre un cambio de modo y un cambio de contexto en un sistema operativo. ¿Por qué el sistema operativo necesita realizar esos cambios?

2. Se tienen 3 procesos: P1, P2 y P3, con tiempos de ejecución: 85, 45 y 118 ms, respectivamente. Si actúa el planificador a largo plazo según el algoritmo SJF (Short Job First) se obtiene que: (Justifique su respuesta)

1. Los procesos se encuentran en la lista de preparados en el orden de llegada: P1, P2 y P3.
2. Los procesos se encuentran en la lista de preparados en el orden: P2, P1 y P3.
3. Los procesos se ejecutan en el orden de llegada: P2, P1 y P3.
4. Los procesos se ejecutan según la prioridad que posean los procesos.

Cambio de modo

1. El sistema operativo distingue dos modos de operación, el modo kernel y el modo usuario, asegurando de esta manera la correcta ejecución del sistema. El hardware tiene agregado un bit de modo el cual indica si se está en el modo usuario o en el modo kernel.

Si se está en el modo usuario, se ejecuta el código definido por este mismo. Si se está en el modo kernel, los privilegios son mayores y se ejecuta el código del sistema operativo.

Este cambio ocurre, por ejemplo, cuando se está en modo usuario y se requiere un servicio del sistema operativo, cambiando así a modo kernel.

Por su parte, el cambio de contexto sucede cuando hay una interrupción y se debe salvar el estado del proceso actual que está corriendo (ya sea en modo kernel o modo usuario), luego de atendida la interrupción se restaura y se continúa con la operación. Esto es realizado por el kernel y se salva el estado en el PCB y se carga el contexto del nuevo proceso a ejecutar.

2. La respuesta correcta es la 2: Los procesos se encuentran en la lista de preparados en el orden: P2, P1 y P3.

Debido a que en la planificación SJF se asocia con cada proceso la longitud de su próxima rafaga de CPU. Cuando el CPU está disponible, se le asigna el proceso que tiene la rafaga mas pequeña. En este caso tenemos:

P1: 85 ms

P2: 45 ms

P3: 118 ms

Por lo cual con SJF nos quedarían en el orden P2, P1 y P3.

Inanición

2.- ¿Puede un sistema detectar inanición? Justifique su respuesta, si es Sí explique cómo lo puede realizar, si es No explique cómo puede manejar este problema.

1.- ¿Puede un sistema detectar inanición? Justifique su respuesta, si es Sí explique cómo lo puede realizar, si es No explique cómo puede manejar este problema.

No, un sistema no puede detectar inanición.

La inanición se produce cuando algún proceso dentro del sistema no puede satisfacer su necesidad de recursos debido a que otros procesos lo están utilizando continuamente.

Esto puede solucionarse implementando algoritmos de planificación que tengan en cuenta el envejecimiento (aging) de procesos. El envejecimiento implica que aquellos procesos que están esperando hace mucho tiempo se les incrementa la prioridad, permitiendo de esta manera desbloquearlo.

1.- Suponga que tiene un sistema en el que hay un ciclo en un grafo de espera del sistema. ¿Es esta una señal definitiva de un interbloqueo? Justifica tu respuesta.

Ciclos en un Grafo de asignación recursos:

Si existe un ciclo en el grafo de espera del sistema no necesariamente existe un interbloqueo, esto depende de la cantidad de recursos disponibles para cada tipo de recurso.

Primer Caso:

En el caso de que cada recurso tenga exactamente una instancia entonces si hay un ciclo existe interbloqueo ya que cada proceso está bloqueado en el ciclo esperando a que otro proceso libere recursos y ningún proceso va a poder obtener los recursos que necesita.

Segundo Caso:

En el caso de que cada tipo de recurso tenga más de una instancia, si existe un ciclo no podemos afirmar que existe un interbloqueo, podría existir un interbloqueo..

1.- Explique cómo se puede implementar un semáforo. Brinde un escenario dónde sea más eficiente utilizar spinlock que locks bloqueantes.

2. ¿Cuál de las siguientes políticas de planificación es más adecuada para un sistema de multiproceso de tiempo compartido? Justifique su respuesta

1. Primero el trabajo más corto.
2. Round-Robin.
3. Prioridades.
4. FIFO.

Semáforo contador: el valor puede variar en un dominio sin restricción.

Semáforo binario: el valor varía entre 0 y 1. Se van a comportar de manera similar a los mutex lock. Los semáforos que no proporcionan mutex lock, pueden utilizar en su lugar semáforos binarios para brindar exclusión mutua.

Un semáforo S es una variable entera a la que, además de la inicialización sólo se accede a través de dos operaciones estándar atómicas, wait y signal.

La operación wait, tiene un ciclo while que se va a ejecutar mientras el valor de S, del semáforo S, sea menor o igual que cero. Una vez que salga del ciclo se va a restar en 1 el valor de ese semáforo. Por el contrario, la operación signal va a sumar en 1 el valor del semáforo.

Este tipo de mutex lock también se denomina spinlock porque el proceso gira mientras que el lock esté disponible. Sin embargo, los spinlock tienen una ventaja fundamental ya que no se requiere un cambio de contexto cuando un proceso espera para obtener un lock, el cual podría demorar un tiempo considerable. En sistemas multicore, los spinlock son una de las mejores opciones para los lock. Si se va a mantener un lock por un periodo corto, un hilo puede girar en un core mientras otro hilo está en su sesión crítica en otro core.

Los que no son spinlock son los que se van a un estado "suspendido" o "bloqueado" cuando quieren hacer wait y tienen que esperar hasta que sean despertados.

otra respuesta:

un programa multi hilado donde se sabe que se van a realizar muchas esperas de corta duración porque al implementarse con busy waiting no se realizan tantos cambios de contexto como se realizarían si se implementaran de la otra forma y al ser esperas de corta duración no estarías desperdiciando tiempo de CPU.

2. Round Robin, dado que se define un intervalo de tiempo denominado "Quantum", cuya duración varía según el sistema. La cola de procesos se estructura como una cola circular y el planificador la recorre asignando un quantum de tiempo a cada proceso.

Garantiza un reparto de la CPU entre todos los usuarios y ahorra tiempos .Round robin, está diseñado especialmente para sistemas de tiempo compartido.

1.- Suponga que tiene un sistema con cuatro hilos (threads), una variable compartida y un lock compartido. El código se escribió de tal manera que cualquier acceso de un hilo a la variable compartida siempre tendría el lock en ese hilo para su acceso. ¿Puede este sistema tener una condición de carrera que involucre la variable compartida? Justifica tu respuesta

2.- Una pareja se va a divorciar. Para dividir sus propiedades, han acordado el siguiente algoritmo. Cada mañana, cada uno puede enviar una carta al abogado del otro solicitando un bien. Dado que se tarda un día en entregar las cartas, han acordado que si ambos descubren que han solicitado el mismo artículo el mismo día, al día siguiente enviarán una carta anulando la solicitud. Entre sus propiedades se encuentran su perro Woofer, la caseta del perro, su canario Tweeter y la jaula del Tweeter. Los animales aman sus casas, por lo que se ha acordado que cualquier división de propiedad que separe a un animal de su casa es inválida, requiriendo que toda la división comience desde cero. Ambos quieren desesperadamente a Woofer. Para que puedan irse de vacaciones (por separado), cada cónyuge ha programado una computadora personal para manejar la negociación. Cuando regresan de vacaciones, las computadoras aún están negociando. ¿Por qué? ¿Es posible un interbloqueo? ¿Es posible inanición? Explique su respuesta

1) NO hay condición de carrera, ya que es excluyente porque todos los hilos utilizan el mismo lock lo cual resulta en que solo uno de ellos a la vez tenga acceso a la variable compartida. La situación de carrera se daría si por ejemplo cada uno tuviera un lock y la variable compartida sigue siendo la misma, ya que en ese caso puede producirse la condición de carrera que varios hilos compiten por obtener esta variable, afectando así el resultado final.

2) Si las computadoras aún están negociando porque cada día eligieron los mismo bienes , un caso puede ser que el primer día ambos eligieron al perro o al canario por lo tanto al día siguiente se inicia la negociación desde cero y sucedió lo mismo durante todas la vacaciones.Otro caso es que se una computadora pida a woofer y la otra la casa del de woofer o que una computadora pida al canario y la otra la jaula del tweeter.

¿Es posible un Interbloqueo? No hay interbloqueo ya que en cualquiera de los casos al día siguiente sabrán si ambos eligieron el mismo bien o uno eligió al animal y otro a su respectiva casa .

¿Es posible inanición?Si es posible ya que si ambos piden los mismo bienes todos los días y se reinicia la división ninguno podrá terminar con su ejecución por lo tanto esta negociación se podrá extender por más tiempo que solo por las vacaciones ya que ningún proceso va a terminar.

1. Si tiene un sistema operativo que está diseñado para una sola CPU y desea adaptarlo a un sistema de múltiples CPU, describa los tipos de cambios / adiciones al software o hardware que necesitaría realizar en relación con el primitivas de sincronización (bloqueos, etc...) y por qué. Liste tres.

2. ¿Cuándo entra un proceso en estado zombie? Justifique su respuesta

1. Cuando muere su padre y él no ha terminado todavía.
2. Cuando muere su padre sin haber hecho wait por él.
3. Cuando él muere y su padre no ha hecho wait por él.
4. Cuando él muere y su padre no ha terminado todavía.

Asumo que las rutinas del sistema operativo tienen un tiempo despreciable, y que tengo suficientes módulos de entrada salida para cada proceso.

3. [10] If you have an OS that's designed for a single CPU, and you want to adapt it to a multi-CPU system, describe the types of changes/additions to the software or hardware you would need to make related to the synchronization primitives (locks, etc...) and why. List three.

The hardware must handle synchronization of shared memory, like making sure swap is atomic.

The scheduler must take into account that moving a thread from one processor to another is expensive. When you pick which thread to wake when you unlock/up this can be taken into account.

On a multi-CPU system, you can use spinlocks much more extensively, because there is another CPU to unlock what you are waiting for.

1.El sistema operativo debe permitir la intercomunicación de procesos a través de memoria compartida, para que estos puedan comunicarse. También se debe asegurar que los procesos no escriban la misma locación de memoria a la vez.

El planificador debe tener el cuenta que mover un hilo de un procesador a otro es costoso.

En un sistema de múltiples CPU los spinlocks pueden ser utilizados más extensamente, ya que hay otro CPU para desbloquear lo que estás esperando.

El planificador debe asegurar que la concurrencia entre procesos no genere problemas o errores en los datos compartidos.

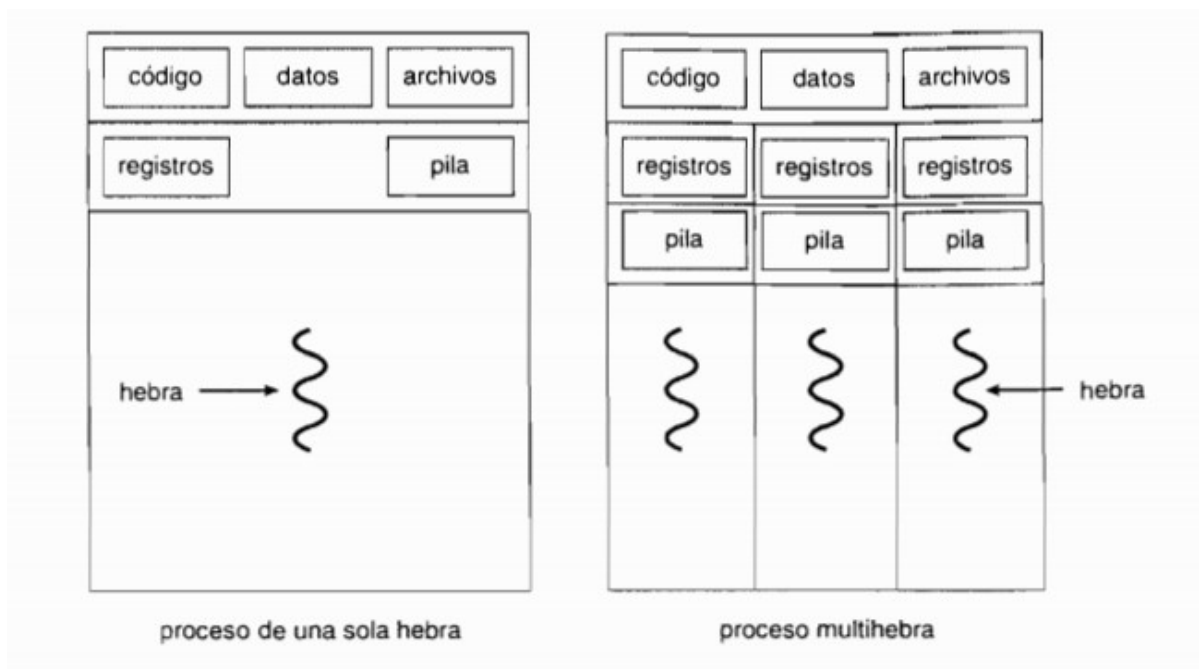
La sincronización entre procesos debe realizarse de forma tal que no haya inanición en los procesos (puede solucionarse con aging) y que no se genere interbloqueo.

2. La opción correcta es la 3, sucede cuando un proceso finaliza pero su padre no ha hecho un wait(), avisándole que su lapso de vida ha terminado, por lo que se convierte en un proceso zombie, ya que continúa teniendo una entrada en la tabla de procesos.

Todos los procesos pasan por este estado cuando terminan, pero es por poco tiempo. Una vez que el padre invoca wait() el identificador del proceso zombie y su entrada en la tabla son liberados.

2. En relación con los threads es cierto que: (Justifique su respuesta)

1. Cuando se crea un thread, su código hay que cargarlo desde el fichero ejecutable a memoria.
F
2. Cada thread tiene su propia pila. V
3. Todos los threads que existen en el sistema en un momento dado, comparten el mismo código. F
4. Descomponer un proceso en dos threads sólo puede ejecutarse más rápido si el sistema cuenta, al menos, con dos procesadores. F



1. Cuando se crea un thread, su código hay que cargarlo desde el fichero ejecutable a memoria:
Falso, ya que los hilos comparten el código correspondiente al proceso que los creó por lo que no es necesario cargarlo desde el fichero ejecutable a memoria.

2. Cada thread tiene su propia pila: **Verdadero, un hilo (thread o proceso de peso liviano) es una unidad básica de utilización de CPU, consiste de un contador de programa, un conjunto de registros y un espacio de pila**

3. Todos los threads que existen en el sistema en un momento dado, comparten el mismo código:
Falso, los hilos comparten el código con el proceso que los crea pero pueden existir diferentes procesos en un mismo sistema por lo que no necesariamente todos los hilos comparten el mismo código.

4. Descomponer un proceso en dos threads sólo puede ejecutarse más rápido si el sistema cuenta, al menos, con dos procesadores: ***no, ya que si se tiene dos hilos a nivel de usuario el sistema operativo los ve como un solo hilo a nivel de kernel por lo tanto no se podrá aprovechar ambos procesadores y se pierde la concurrencia.***

1. Brinde un ejemplo para distinguir entre un proceso bloqueado y un proceso interbloqueado.
2. ¿Cuál de estas transiciones de estados de un proceso jamás se produce en un sistema normal? Justifique su respuesta
 1. de bloqueado a listo.
 2. de listo a bloqueado.
 3. de ejecutando a listo.
 4. de ejecutando a bloqueado.

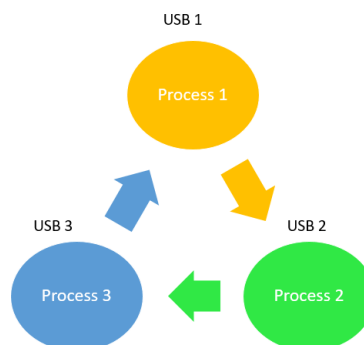
1. Ejemplo:

Proceso bloqueado: un proceso P1 está bloqueado cuando otro proceso P0 tiene un recurso que P1 necesita. Una vez que P0 libera el recurso o termina su ejecución P1 puede continuar ejecutando.

Por ejemplo el proceso P1 quiere imprimir pero P0 está usando la impresora, cuando P0 termine de imprimir, el siguiente en la cola será P1 y podrá imprimir.

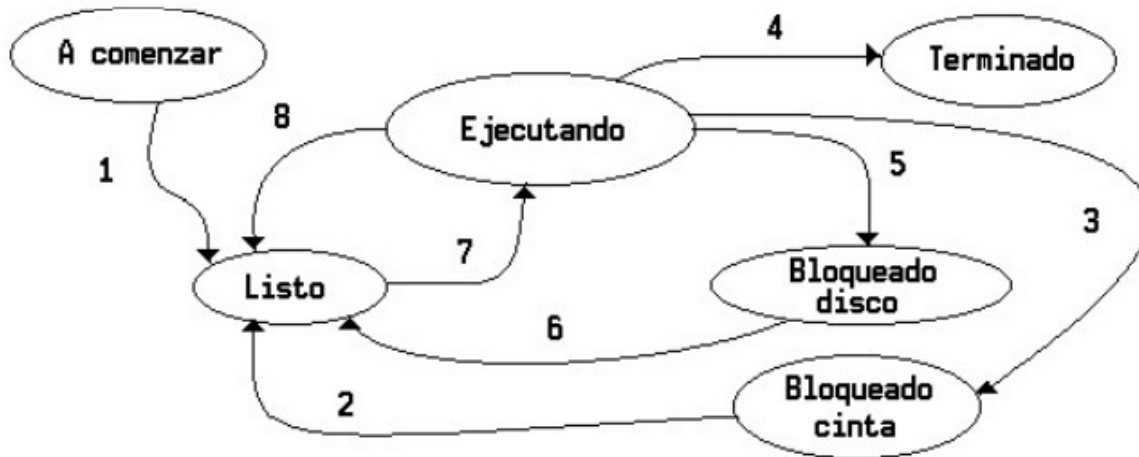
Proceso interbloqueado: un proceso P1 y P2 que están interbloqueados sucede cuando P1 tiene un recurso que P2 necesita y P2 necesita un recurso que tiene P1, por lo cual se bloquean entre ellos y no pueden continuar.

Por ejemplo una computadora tiene tres puertos USB y tres procesos. Cada uno de estos procesos tiene un puerto USB. Cuando cada proceso requiere otro recurso, los tres procesos caerán en deadlock ya que cada uno estará esperando a que el otro libere el recurso que está utilizando. Esto resulta en una espera circular.



2. De Listo a bloqueado

No puede pasarse de listo a bloqueado ya que en situaciones normales un proceso pasa a estar bloqueado porque requiere un recurso que no está disponible, pero si está en la cola de listos el proceso está esperando y no realiza acción alguna más que esa.



1.- ¿Qué significa espera ocupada (busy waiting)? ¿Qué tipos de espera ocupada hay en un sistema operativo? ¿Se puede evitar por completo? Explique su respuesta.

2.- La siguiente solución del problema Productor/Consumidor es correcta. Detalle su respuesta.

semaphore mutex = 0, empty = 0;

<pre> void Producer() { down(mutex); produce(); up(empty); up(mutex); } </pre>	<pre> void Consumer() { down(mutex); down(empty); consume(); up(mutex); } </pre>
--	--

1. El busy waiting es cuando un proceso espera y continuamente chequea la condición que lo hace esperar. Esto desperdicia ciclos del CPU que otros procesos podrían usar productivamente. Esto es un problema en multiprogramación, donde una sola CPU se comparte entre todos los procesos.

Busy waiting means that a process is waiting for a condition to be satisfied in a tight loop without relinquishing the processor. Alternatively, a process could wait by relinquishing the processor, and block on a condition and wait to be awakened at some appropriate time in the future. Busy waiting can be avoided but incurs the overhead associated with putting a process to sleep and having to wake it up when the appropriate program state is reached.

<https://www.os-book.com/OSE2/practice-exer-dir/5-web.pdf>

2. No es correcta la solución, porque el mutex es inicializado en 0 y además que ambos en su primera línea ejecuten `down(mutex)` es incorrecto.

Si el mutex está en 0, tanto el productor como el consumidor al hacer `down(mutex)` se quedarán esperando.

En el caso de tener el mutex inicializado en 1, y empty en 0. En el caso de que el consumidor realice `down(mutex)` cuando quiera hacer `down(empty)` no podrá realizarlo ya que el productor no produjo. Por esto el consumidor queda a la espera, y el productor también queda bloqueado ya que el consumidor tiene el mutex.

1.- ¿Por qué la planificación de colas multinivel con retroalimentación varían el quantum de tiempo asociado con diferentes colas? ¿Cuál sería la desventaja de tener el mismo quantum de tiempo en todos los niveles?

2.- ¿Qué es falso acerca del paso de mensajes? Justifique su respuesta

1. Implica una sección crítica en los datos del mensaje.
2. Una recepción bloqueante implica sincronización con el emisor.
3. Un envío bloqueante permite al emisor utilizar el buffer del mensaje cuando se desbloquea.
4. El paso de mensajes se implementa siempre con memoria compartida.

1) La planificación de colas multinivel con retroalimentación varía el quantum de tiempo asociado a las diferentes colas para atender con mayor prioridad a los procesos más cortos (ej. aquellos que requieren de una E/S o que requieren poco tiempo de CPU y se ejecutan con mucha frecuencia), esto ocurre porque las colas de mayor prioridad tienen un quantum lo suficientemente chico para que los procesos que requieran menor tiempo de CPU completen su ejecución, mientras que los que requieran mayor tiempo caen a un nivel de menor prioridad. Teniendo el mismo quantum de tiempo en todos los niveles no habría esta distinción de priorizar a los procesos que requieren menor tiempo de CPU y/o se ejecutan con mayor frecuencia.

2) 4. Falso. Los pasos de mensajes, además de implementarse con memoria compartida, es posible implementarlos con mecanismos de comunicación directa o indirecta (mailbox). Otra forma es haciendo uso de pipes.

1.- Considere un proceso compuesto por dos hilos, *p* y *q*, que están compuestos por un conjunto de sentencias atómicas (A, B, C, D, y E). El proceso crea los dos hilos y comienza la ejecución concurrente. Muestre todos los posibles entrelazados de ejecución, considerando que los hilos contienen las siguientes sentencias.

<code>void p()</code>	<code>void q()</code>
<code>{ A;</code>	<code>{ D;</code>
<code> B;</code>	<code> E;</code>
<code> C;</code>	<code>}</code>
<code>}</code>	

2.- ¿Puede un sistema detectar inanición? Justifique su respuesta, si es Sí explique cómo lo puede realizar, si es No explique cómo puede manejar este problema.

2) No no es posible detectar la inanición en un sistema, es por esto que se debe evitar implementando mecanismos como, por ejemplo, en la planificación por prioridad, aumentar la prioridad de un proceso a medida que pasa el tiempo y no es atendido (envejecimiento). En el caso

del mecanismos de recuperación de interbloqueo "apropiación de recursos", para evitar la inanición de un proceso por ser elegido constantemente como víctima para hacerle rollback, se podría incorporar la cantidad de rollbacks hechos a un proceso como factor a la hora de elegir una víctima logrando que no sea un mismo proceso el que siempre se elige.

1. Si tiene un sistema operativo que está diseñado para una sola CPU y desea adaptarlo a un sistema de múltiples CPU, describa los tipos de cambios / adiciones al software o hardware que necesitaría realizar en relación con la administración de procesos y por qué.

Respuesta:

If multiple CPUs are available, load sharing, where multiple threads may run in parallel, becomes possible, however scheduling issues become correspondingly more complex

One approach to CPU scheduling in a multiprocessor system has all scheduling decisions, I/O processing, and other system activities handled by a single processor — the master server. The other processors execute only user code. This asymmetric multiprocessing is simple because only one core accesses the system data structures, reducing the need for data sharing. The downfall of this approach is the master server becomes a potential bottleneck where overall system performance may be reduced. The standard approach for supporting multiprocessors is symmetric multiprocessing (SMP), where each processor is self-scheduling. Scheduling proceeds by having the scheduler for each processor examine the ready queue and select a thread to run. Note that this provides two possible strategies for organizing the threads eligible to be scheduled:

1. All threads may be in a common ready queue.
2. Each processor may have its own private queue of threads.

Tipo de planificación en los sistemas multiprocesador

Multiprocesamiento Asimétrico (Asymmetric)

Solo un procesador (el master) accede a la estructuras de datos del sistema, simplificando y reduciendo, el manejo de datos compartidos.

Tiene una desventaja muy importante y es que el servidor maestro se convierte en un potencial cuello de botella y eso impacta en la performance, ya que todos los núcleos, le van a pedir los datos a un solo núcleo.

Multiprocesamiento Simétrico (Symmetric SMP)

Cada uno de los procesadores puede realizar la planificación para sí mismo. Hay dos estrategias que se pueden seguir: todos los hilos pueden estar en una cola en común o cada procesador tiene su propia cola de hilos.

Afinidad al procesador:

Cuando un proceso se ha estado ejecutando en un procesador espacio los datos a los que el proceso ha accedido más recientemente se almacenan en la caché del procesador y, como resultado, los sucesivos accesos a memoria por parte del proceso suelen satisfacerse si más que consultar la memoria caché. Si el proceso migra a otro procesador los contenidos de la memoria caché del procesador de origen deben invalidarse y la caché del procesador destino debe ser rellenada. Debido al alto coste de invalidar y rellenar las memorias caché, la mayoría de los sistemas SMP intentan evitar la migración de procesos de un procesador a otro, y en su lugar intentan mantener en ejecución cada proceso en el mismo procesador. Esto se conoce con el nombre de **afinidad al procesador**, lo que significa que un proceso tiene una afinidad hacia el procesador en que está ejecutándose actualmente.

La afinidad al procesador toma varias formas. Cuando un sistema operativo tiene la política de intentar mantener un proceso en ejecución en el mismo procesador, pero no está garantizando que lo hagan nos encontramos ante una situación conocida como **afinidad suave**. En este caso, es posible que un proceso migre entre procesadores. Algunos sistemas, como linux, también proporcionan llamadas al sistema que soportan **afinidad dura**, la cual permite a un proceso especificar que no debe migrar a otros procesadores.

Equilibrio de carga:

Los mecanismos de equilibrado de carga intentan distribuir equitativamente la carga de trabajo entre todos los procesadores del sistema SMP, en los sistemas con una cola de ejecución común el equilibrado de carga no es necesario, ya que una vez que un procesador pasa a estar inactivo inmediatamente extrae un proceso ejecutable de la cola de ejecución común. Es importante destacar que en la mayoría de los sistemas operativos cada procesador tiene una cola privada de procesos listos, existen 2 métodos generales para equilibrar la carga:

Migración comandada (push migration): con la migración comandada una tarea específica comprueba periódicamente la carga de cada procesador y si encuentra un desequilibrio distribuye equitativamente moviendo procesos de los procesadores sobrecargados en los menos ocupados o inactivos.

Migración solicitada (pull migration): la migración solicitada se produce cuando un procesador inactivo extrae de un procesador ocupado alguna tarea que esté en espera.

Estos dos métodos no tienen porque ser mutuamente excluyentes y de hecho a menudo se implementan en paralelo en sistemas de equilibrado de carga, como por ejemplo el planificador de Linux.

Mecanismo de multi hilado:

Se debe incluir una estrategia de mecanismo multi hilado (SMT).

La idea es crear varios procesadores lógicos sobre un mismo procesador físico, y presentar una vista de varios procesadores lógicos al sistemas operativo.

SMT es una funcionalidad proporcionada por hardware, ya que se debe proporcionar la representación del estado de la arquitectura de cada procesador lógico (incluye los registros de propósito general y los registros de estado de la máquina), y el tratamiento de interrupciones.

Pregunta 3
Sin responder aún
Puntúa como 40,00
¡¹! Marcar pregunta

1.- ¿Puede un sistema detectar inanición? Justifique su respuesta, si es Sí explique cómo lo puede realizar, si es No explique cómo puede manejar este problema.

2.- Considere el siguiente código para testear locks y variables de condición. Comienza cuando el hilo "principal" llama a ThreadTest (). Realice una traza de ejecución de este programa hasta que imprima el mensaje "Pare aquí". Considere que la planificación es FCFS y las colas se administran utilizando FIFO.

Lock *l;

Condition *cv;

<pre>void A(int arg) { l->Acquire(); cv->Signal(); Thread->Yield(); cv->Wait(); l->Release(); } void B(int arg) { l->Acquire(); cv->Wait(); Thread->Yield(); cv->Signal(); l->Release(); }</pre>	<pre>void ThreadTest() { Thread *t; l = new Lock("lock"); cv = new Condition("cv"); createThread("A"); createThread("B"); Thread->Yield(); Thread->Yield(); printf("Pare aquí\n"); }</pre>
---	--

Thread->yield obliga al hilo que realiza la llamada a renunciar al uso del procesador y a esperar en la cola de listos.

1. Escribir la secuencia de cambios de contexto que ocurrieron hasta este punto,
2. Enumerar las colas en las que se encuentran los hilos en este punto, y su orden relativo si hay más de un hilo en una cola
3. Indicar el estado de cada uno de los hilos.

1.- Se tienen tres procesos cuyo comportamiento se establece en la tabla que hay a continuación. "Llegada" representa el momento en el que el proceso llega al sistema, los valores que hay en las columnas etiquetadas por CPU son las unidades de tiempo de la ráfaga de CPU del proceso. Con E/S representa el tiempo que requiere su entrada salida.

Proceso	T. Arribo	CPU	E/S	CPU	E/S	CPU	E/S	CPU
A	0	10	15	5	20	10	-----	-----
B	5	5	20	15	10	10	10	10
C	10	10	5	10	5	25	10	5

Mostrar en un diagrama de Gantt la ejecución de estos procesos y obtener el tiempo retorno y de espera para cada uno de los procesos considerando las siguientes planificaciones:

1. FIFO
2. Trabajo más corto primero (SJF)

Si hay solo una instancia de un tipo de recurso se realiza el grafo de asignación de recursos. Para múltiples instancias se utiliza el algoritmo del banquero.

Cualquier solución al problema de la sección crítica deberá satisfacer los tres requisitos siguientes:

1. **Exclusión mutua:** Si el proceso P_j está ejecutando su sección crítica, los demás procesos no pueden estar ejecutando sus secciones críticas.

2. **Progreso:** Si ningún proceso está ejecutando su sección crítica y algunos procesos desean entrar en sus correspondientes secciones críticas, sólo aquellos que no estén ejecutando sus acciones restantes pueden participar de cuál será el siguiente que entre en su sección crítica, y esta selección no se puede posponer indefinidamente.
3. **Espera limitada:** Existe un límite en el número de veces que se permite que otros procesos entren en sus secciones críticas después de que un proceso haya hecho una solicitud para entrar en su sección crítica y antes de que la misma haya sido concedida.
Estamos suponiendo que cada proceso se ejecuta a una velocidad distinta de cero. Sin embargo, no podemos hacer ninguna suposición sobre la velocidad relativa de los n procesos.

1.- Suponga que la operación `Signal()` en una variable de condición no garantiza despertar solo un hilo, y no garantiza que los despierte en orden FIFO. ¿Podría esto afectar a los programas que utilizan variables de condición? Explique su respuesta, brinde un ejemplo de un programa que fallaría usando esta semántica más débil.

2.- La siguiente es una propuesta de solución basada en software para el problema de exclusión mutua entre dos procesos.

`boolean flag[2] = {true, false};`

P0: repeat while (flag[1]); flag[0] = true; Sección Crítica	P1: repeat while (flag[0]); flag[1] = true; Sección Crítica
---	---

1. Si puede afectar

En el caso de despertar más de un solo hilo y no tener FIFO

Puede suceder que:

P0	P1
cv->wait()	cv->wait()
Lock->acquire()	Lock->acquire()
----	---
Lock->release()	Lock->release()

P2 hace un signal y P0 y P1 toman la señal

Uno solo de ellos adquirirá el lock, y puede perderse la sincronización de quien debía tomar el lock primero, resultando en un resultado erróneo y posible deadlock.

Ejercicio 1

- 1.- Enumere ventajas y desventajas de kernels monolítico, micro-kernel y modulados.
- 2.- Describa dos operaciones del Sistema Operativo para proteger los recursos.

El **kernel monolítico** ubica la funcionalidad del kernel en un archivo binario único y estático que corre en un único espacio de memoria.

Es difícil de implementar y extender, hay un gran consumo de recursos para las llamadas al sistema.

Es un modelo rápido y eficiente debido a que la comunicación se realiza sin necesidad de enviar mensajes, sino que la comunicación de los procesos se realiza por dentro del kernel. (por dentro porque es todo un archivo y por ende sería un solo).

Tiene una veloz comunicación entre el kernel. Es un modelo rápido y eficiente.

El microkernel estructura el sistema operativo removiendo todos los componentes no esenciales del kernel, e implementandolos como programas a nivel de usuario que residen en diferentes espacios de memoria.

El resultado es un kernel pequeño. Su función principal es permitir la comunicación entre el programa cliente y los distintos servicios que están corriendo en el espacio del usuario.

La comunicación se realiza mediante el pasaje de mensajes.

Ventaja: hace que el sistema operativo sea más fácil de extender, ya que todos los nuevos programas son agregados al espacio del usuario. Además es fácil de portar de un diseño de hardware a otro. Provee mayor seguridad y confianza, ya que como la mayoría de los servicios se encuentran corriendo como procesos usuarios si el servicio falla el resto del sistema operativo permanece sin alterar.

Desventaja: La performance de los microkernels puede verse afectada debido a un consumo elevado en las funciones del sistema. Esto debido a que para la comunicación de servicios a niveles de usuario se deben copiar los mensajes entre los servicios, los cuales residen en distintos espacios de memoria. Además el sistema operativo debe cambiar de un proceso a otro para intercambiar los mensajes.

Modulados: en este tipo de sistemas operativos, la estructuración se realiza en forma de módulos bien definidos, donde cada módulo tiene una funcionalidad específica. El kernel provee funcionalidades esenciales, y luego puede llamar dinámicamente a cada módulo. Cada módulo se puede comunicar con cualquiera de los otros módulos del sistema. Los módulos no necesitan comunicarse mediante paso de mensajes.

Ventajas: confiabilidad, ya que si un módulo falla no afecta al resto. Además, su implementación y extensión es más fácil que la de un sistema monolítico donde el sistema se ubica en un mismo archivo, y es más eficiente que un microkernel dado que la comunicación entre módulos no necesita pasaje de mensajes..

Desventajas no encontré en el libro

Datazo: La mayoría de los sistemas operativos usan ese.

Código Monitores

1. N tamaño del buffer

```
monitor prodCons{
    condition lleno,vacio;
    int cant;
    agregar{
        if(cant == n)
            lleno.wait //Si está lleno hago esperar al productor
        cant++; //introducir al buffer
        if(cant == 1)
            vacio.signal //Si estaba vacio aviso al consumidor que hay algo
    }
    eliminar{
        if(cant==0)
            vacio.wait //Si está vacio hago esperar al consumidor
        cant--; //saco del buffer
        if(cant==n-1)
            lleno.signal //Si antes estaba lleno aviso al productor que ya no
    }
}
```

HILOS

primero tenes q pensar en la

Indice resúmenes eli:

definición de hilo y componentes

beneficios de la programación multihilo

programación multinúcleo (diferencia entre concurrencia y paralelismo)

desafíos de la programación multinúcleo

tipos de paralelismo (de datos y tareas)

Modelos multihilo (uno a muchos, etc) ventajas y desventajas

Hilado implícito

Fork Join

problemas de Hilado

Manejo de señales

Cancelación de hilos

(las ventajas de usar hilos está en la pregunta 14 del tp2)

(hilos a nivel de kernel vs usuario tmb esta ahi)

PROCESOS

Concepto y PCB

Estados y creación

comunicación entre procesos (IPC)

-memoria compartida

-pasaje de mensajes

comunicacion sincronica y asincronica

buffer automático o explícito

PLANIFICACIÓN DE PROCESOS:

Planificación Apropiativa y no apropiativa

Dispatcher

criterio de planificación

planificación de hilos

planificación local (a nivel de usuario)

planificación global (a nivel de kernel)

planificación de múltiples procesadores

master server

multiprocesamiento asimétrico

multiprocesamiento simétrico

equilibrio de carga

afinidad del procesador

multiprocesamiento heterogéneo

Planificación en tiempo real

-sistemas en tiempo real blandos (solo les da prioridad a los procesos)

-sistemas en tiempo real estrictos (tienen un tiempo límite, no cumplirlo es lo mismo que no hacerlo)

Minimización de latencia